

AI Daily Academy

day 003 2026-08-15

通勤用音声教材の文字版 - 最新情報の参照元は末尾に掲載

この音声はAI生成音声でお届けします。

AI Daily Academy、Week 1「AIの全体像」、Day 3を始めます。

今日のテーマは、「Embeddingとベクトル」です。

昨日まで、私たちは生成AIとLLM、そしてトークンとコンテキストについて学びました。今日は、その続きです。AIが文章を「似ている」「関係がある」と判断して、必要な情報を探し出すための基礎技術、Embedding、つまり埋め込み表現について理解していきます。

少し数学っぽく聞こえるかもしれませんが、心配はいりません。通勤中に聞きながら、「AIは言葉を地図の上の位置に変えているんだな」とつかめれば、今日の目的は達成です。

それでは始めましょう。

まず、前回の復習です。

Day 1では、生成AIとLLMについて学びました。LLMは、大量の文章から言葉のつながり方を学習し、次に続きそうな言葉を予測することで、自然な文章を作る仕組みでした。

Day 2では、AIが文章をそのまま読んでいるわけではなく、トークンという細かな単位に分けて処理していることを学びました。そして、一度の会話や指示の中でAIが参照できる情報量には、コンテキスト、つまり文脈の窓という上限があることも確認しました。

ここで一つ、重要な疑問が出てきます。

会社には数千件のマニュアルがある。過去の議事録もある。商品カタログもある。社内ルールもある。その中から、「今の質問に関係する情報」をAIはどうやって見つけるのでしょうか。

単語がぴったり一致する文書だけを探せばよいのでしょうか。

たとえば、「出張の交通費を精算したい」と質問した人に対して、社内規程には「旅費精算の申請期限は帰着後10営業日以内」と書いてあるかもしれません。「交通費」と「旅費」は文字としては違います。でも、人間なら意味が近いと分かります。

この「言葉が違ってても、意味が近いことを扱う」ための中心技術が、今日のEmbeddingです。

Embeddingは、日本語では「埋め込み表現」と訳されます。少し難しい表現ですが、ここではこう覚えてください。

Embeddingとは、文章や単語、画像などの意味や特徴を、たくさんの数字の並びに変換する技術です。

そして、その数字の並びをベクトルと呼びます。

ベクトルというと、高校数学の矢印を思い出す方もいるかもしれません。AIでも基本の考え方は似ています。ベクトルは、複数の数字をひとまとまりにしたものです。

たとえば、ある文章を、仮に三つの数字で表すとします。

「犬が公園で走る」という文章が、0.8、0.2、0.6。

「子犬が芝生を駆け回る」という文章が、0.75、0.25、0.58。

「経費精算の申請方法」という文章が、マイナス0.4、0.9、マイナス0.1。

これは説明のための単純な例です。実際には、数百個から数千個以上の数字を使うことが一般的です。

大切なのは、数字そのものを人間が読んで意味を理解する必要はない、という点です。

「0.8は犬らしさで、0.2は公園らしさ」と、一つひとつの数字に分かりやすい名前が付いているわけではありません。たくさんの数字の組み合わせ全体によって、意味の特徴が表現されています。

AIは学習を通じて、意味や使われ方が近い言葉、似た場面で登場する文章を、ベクトル空間の近い場所に配置するようになります。

ベクトル空間という言葉も出てきましたね。

これは、たくさんの数字で作られた巨大な地図のようなものだと思います。

「犬」「猫」「動物」「ペット」といった言葉は、意味が近いので地図上でも近くに集まりやすい。

「請求書」「見積書」「経費精算」「領収書」は、仕事や会計に関する言葉として近い場所に集まりやすい。

一方で、「南極の気温」と「今月の広告予算」は、通常は遠い場所に配置されやすい。

ここで誤解してはいけないことがあります。

Embeddingは、辞書に載っている正確な定義だけを見て、言葉を分類しているわけではありません。大量の文章の中で、どの言葉がどの言葉と一緒に使われるか、どんな文脈で登場するか、といったパターンから、意味の近さを捉えます。

だから、「スマートフォン」と「携帯電話」、「採用」と「人材募集」、「解約」と「退会」のように、表記が違ってても関係を見つけられる可能性があります。

ただし、ここで「AIは意味を完璧に理解している」と考えるのは早すぎます。

Embeddingは、学習データに含まれる傾向をもとに、意味の近さを数字として表している仕組みです。専門用語、社内だけで通じる略語、文脈によって意味が変わる言葉では、期待通りに近さを表せない場合もあります。

たとえば「リード」という言葉は、営業では見込み顧客、音楽では主旋律、犬の散歩ではひもを指すことがあります。前後の文脈が少なければ、AIがどの意味として扱うかは不安定になることがあります。

では、ベクトル同士の「近さ」はどう測るのでしょうか。

代表的な方法の一つが、コサイン類似度です。

難しい名前ですが、覚えるべきポイントは一つだけです。

二つのベクトルがどれくらい同じ方向を向いているかを測り、意味の近さの目安にする方法です。

数字が大きいほど似ている、と単純に考えるよりも、「向きが似ているか」を見るほうが便利な場面があります。文章の長さが違っていても、意味の方向性が近いかを比較しやすいからです。

たとえば、「有給休暇はいつまでに申請しますか」と、「休暇届の締切を教えてください」は、使っている言葉が完全には一致しません。しかし意味の方向は近い。Embeddingでベクトルにして比較すると、高い類似度になることが期待できます。

逆に、「休暇中の自動返信メールの書き方」は、同じ休暇という言葉を含んでいても、知りたい内容は申請ルールとは違います。単なるキーワード一致よりも、質問全体の意味で探することが重要です。

これが、検索にEmbeddingを使う大きな理由です。

従来のキーワード検索は、入力した文字列と一致する語を中心に探するのが得意です。商品コード、固有の型番、法令番号、正確な製品名を探すときには、とても有効です。

一方、ベクトル検索は、意味が近い文章を探すのが得意です。言い換え、曖昧な質問、自然な話し言葉から、関連する情報を探す場面で力を発揮します。

実務では、この二つを競争させるのではなく、組み合わせることが多くなります。

たとえば、「AB-123という製品の返品条件」という検索なら、製品番号の一致は非常に重要です。ここはキーワード検索が強い。

一方で、「買った商品を返したい。手続きはどうすればいいですか」という質問なら、「返品」「返金」「キャンセル」「交換」といった意味の近さを扱えるベクトル検索が役立ちます。

最近よく使われる構成に、ハイブリッド検索があります。

これは、キーワード検索とベクトル検索の両方で候補を出し、その結果を組み合わせる方法です。正確な文字列の一致と、意味の近さの両方を取り込めるため、社内検索や顧客向けFAQでよく検討され

ます。

ここまでを、AIの動きとして順番に見てみましょう。

社員が社内チャットで、「来週の大阪出張、ホテル代の上限はいくらですか」と質問したとします。

まず、その質問文をEmbeddingモデルに渡します。

Embeddingモデルは質問をベクトルに変換します。

次に、あらかじめベクトル化して保存しておいた社内規程の文章群から、質問ベクトルに近いものを探します。

たとえば、「国内出張における宿泊費の上限額は、地域区分に応じて定める」という規程の一節が見つかるかもしれません。

さらに、その前後の文章も一緒に取り出します。「大阪はどの地域区分か」「例外申請の条件は何か」といった情報です。

最後に、LLMへ質問と関連資料を渡し、「以下の社内規程だけを根拠に、結論と条件を簡潔に教えてください」と指示します。

この流れが、RAGと呼ばれる仕組みの基本です。

RAGは、Retrieval-Augmented Generationの略です。日本語にすると、「検索で取り出した情報を補強材料にして、文章を生成する」といった意味です。

重要なのは、RAGはLLMに新しい知識を永久に覚え込ませる方法ではない、ということです。

質問のたびに必要な資料を検索して、その場でLLMに読ませる方法です。

Day 2で学んだコンテキストを思い出してください。LLMは、一度に読める情報量に上限があります。何万件もの規程を全部渡すことは、コスト面でも精度面でも現実的ではありません。

そこでEmbeddingを使い、「この質問に関係がありそうな数件だけ」を先に選びます。選んだ文章をコンテキストに入れ、LLMに回答させます。

つまり、Embeddingは、LLMの前に立つ優秀な司書のような役割です。

LLMが回答を書く人だとすれば、Embeddingとベクトル検索は、膨大な書庫から関連資料を素早く探して机の上に並べる人です。

この役割分担を理解すると、生成AIを使った検索システムの全体像が見えやすくなります。

ここで、もう一つ重要な考え方があります。それがチャンクです。

チャンクとは、長い文書を検索しやすい大きさに分けた一つひとつの断片です。

たとえば、100ページある就業規則を、文書全体で一つのベクトルにしてしまうとどうなるでしょうか。休暇、給与、退職、服務規律、育児支援など、たくさんのテーマが混ざります。質問との近さがぼやけてしまいます。

逆に、一文ずつ細かく分けすぎると、必要な前提や例外条件が別の断片に散らばり、意味が切れてしまいます。

そこで、意味のまとまりを考えながら、数段落ごと、あるいは見出しごとに分けます。これがチャンク設計です。

たとえば「出張旅費規程」という見出しの下に、宿泊費、交通費、日当、申請期限が書かれているなら、テーマごとに適切なサイズへ分けます。そして、各チャンクにタイトル、章番号、更新日、対象部署、文書URLなどの情報を付けます。

この付加情報をメタデータと呼びます。

メタデータは、検索の精度と安全性を高めるために非常に重要です。

たとえば、人事部向けの非公開規程と、全社員向けの公開規程が混在している場合を考えてください。質問者の権限に応じて、検索対象を絞り込まなければなりません。

ここで、「人事部限定」「2026年度版」「日本法人向け」といったメタデータを使います。

Embeddingが意味の近さを探す役割なら、メタデータは「誰が、どの範囲の、いつの情報を見てよいか」を管理する役割です。

実務では、ベクトル検索の精度だけに注目しがちですが、アクセス権、版管理、更新日、出典表示まで設計して初めて、信頼できる仕組みになります。

では、身近な具体例で考えてみましょう。

一つ目は、動画や音楽のおすすめです。

あなたが、落ち着いたアコースティック音楽をよく聴いているとします。サービスは、曲名が同じかどうかだけでなく、テンポ、楽器、雰囲気、聴かれ方などの特徴を数値化し、似た曲を提案することがあります。

二つ目は、写真アプリです。

「海の写真を見せて」と検索すると、ファイル名に「海」と書いていない写真まで候補に出ることがあります。画像の中に空、砂浜、波、船などの特徴があることをAIが捉え、画像をベクトルとして扱っている場合があります。

三つ目は、ECサイトの商品検索です。

「雨の日に使える通勤バッグ」を探したとき、商品説明にそのまま同じ言葉がなくても、「防水」「撥水」「PC収納」「ビジネス向け」といった内容から、近い商品を探せるようになります。

ただし、おすすめや検索は便利な反面、過去の行動やデータの偏りを反映することがあります。「似ている」という判定は、客観的な真実そのものではなく、モデルとデータによる計算結果です。検索結果の上位に出たからといって、それが唯一の正解だとは限りません。

次に、実務での活用例を見ていきます。

一つ目は、社内ナレッジ検索です。

総務、人事、営業、開発、法務などの資料を横断して、「この手続きの担当は誰か」「過去に似た提案書はあるか」「この顧客の業界向けの事例は何か」と自然な言葉で探せるようにします。

ただし、最初から全社データを入れるのではなく、まずは公開範囲が明確で、更新管理できるFAQや手順書から始めるのが現実的です。

二つ目は、問い合わせ対応の支援です。

顧客の問い合わせをEmbeddingに変換し、過去の解決済み問い合わせやヘルプ記事から近いものを探します。その上で、担当者に回答案を提示します。

ここで大事なのは、AIにいきなり自動返信を任せるのではなく、最初は人が確認して送る設計にすることです。特に、料金、契約、医療、金融、法務、個人情報に関わる回答では、誤案内の影響が大きくなります。

三つ目は、営業支援です。

商談メモ、提案資料、成功事例をベクトル化し、「この顧客の課題に近い導入事例を探して」と検索します。営業担当者がゼロから資料を探す時間を減らせます。

ただし、顧客情報を外部サービスに送る場合には、契約、利用規約、データ保存先、学習利用の有無、社内ルールを必ず確認する必要があります。技術的にできることと、業務上やってよいことは別です。

四つ目は、文書の重複発見です。

似た内容の規程、古いマニュアル、ほぼ同じ提案資料を見つける用途です。完全に一致しないけれど、内容が重なっている文書を候補として出せます。整理の入口として有効です。

では、実装のイメージです。

今日の目的は、長いコードを暗記することではありません。Embeddingを使った検索の処理を、頭の中で組み立てられるようになることです。

処理は、大きく五段階です。

第一段階。検索対象の文書を集めます。

たとえば、社内FAQの回答本文、タイトル、更新日、公開範囲を用意します。

第二段階。文書をチャンクに分けます。

FAQなら、一つの質問と回答を一つのチャンクにできるでしょう。長い規程なら、見出し単位や段落単位で分けます。

第三段階。各チャンクをEmbeddingモデルでベクトル化し、ベクトルデータベースに保存します。

ベクトルデータベースとは、ベクトル同士の近さを高速に探すことに向いたデータベースです。通常のデータベースでも保存はできますが、数万件、数百万件と増えたときに、近いベクトルを素早く探す仕組みが必要になります。

この高速な近似検索は、ANN検索と呼ばれることがあります。Approximate Nearest Neighbor、つまり「完全に総当たりではないが、十分に近い候補を高速に探す方法」です。

第四段階。利用者の質問を同じEmbeddingモデルでベクトルに変換します。

ここで重要なのは、原則として文書側と質問側で互換性のある同じEmbeddingモデルを使うことです。異なる基準で作った地図同士を比べても、距離の意味が崩れる可能性があるからです。

第五段階。近いチャンクを数件取得し、必要ならキーワード検索の結果も合わせます。その資料と質問をLLMに渡し、出典付きで回答を生成させます。

擬似コードにすると、考え方はこうです。

「文書を分割する。各断片をベクトル化する。ベクトルとメタデータを保存する。質問をベクトル化する。近い断片を検索する。断片だけを根拠としてLLMに答えさせる。」

実装で最も見落とされやすいのは、評価です。

検索結果がもっともらしく見えても、本当に必要な資料を取り出せているとは限りません。

そこで、実際に利用者が聞きそうな質問を、まず二十件から五十件ほど用意します。そして、それぞれについて「本来参照されるべき文書」を人が決めます。

その質問で検索し、正しい文書が上位に出るかを確認します。

もし出なければ、原因を切り分けます。

チャンクが大きすぎるのか。小さすぎるのか。文書のタイトルが含まれていないのか。メタデータの絞り込みが強すぎるのか。キーワード検索との併用が必要なのか。そもそも元文書が古いのか。

AIプロジェクトでは、モデル選びだけで成果が決まるわけではありません。データの整理、文書の更新、検索の評価、利用者の確認手順が、同じくらい重要です。

ここで、今日のAI最新情報です。2026年8月15日時点で確認できる、公式発表を中心に、Embeddingとベクトルの学びにつながる話題を四件紹介します。

一件目。8月14日、Anthropicは、Claudeが生成するテキストへの透かし技術について説明を公開しました。発表によると、将来のClaudeモデルでは、生成文に人間には見分けにくい統計的なパターンを持たせ、Claudeが関与した可能性を判定できるようにする方針です。

今日のテーマとの関係は、AIが文章をそのまま文字列としてだけでなく、確率や数値的な特徴として扱っている点です。ただし、透かしはEmbeddingやベクトル検索とは別の技術です。Embeddingが意味の近さを扱うのに対し、透かしは生成元を確認するための手がかりを扱います。目的が違うことを区別しておきましょう。

二件目。8月12日、OpenAIは、企業でのAI活用が、質問への回答支援から、複数の手順を実行するエージェント型の仕事へ移りつつあるとするレポートを公開しました。そこで強調されているのは、AIが実務を進めるには、社内の文脈、利用できるツール、権限、人によるレビューが必要だということです。

これはRAGとベクトル検索の重要性に直結します。AIエージェントが仕事をするためには、一般論を答えるだけでなく、その会社の正しい資料を探し、許可された範囲で使える必要があります。Embeddingは、その「必要な社内文脈を探す」基盤の一つです。

三件目。7月23日、Googleは、人々がAIを仕事や日常でどのように使っているかを分析するATLASの初回報告を公開しました。Googleの説明では、集計・匿名化された大規模な人とAIのやり取りを、構造化して分析しています。

この種の大規模な文章データを整理し、似た利用目的やタスクを分類する際にも、文章を数値化し、近さやまとまりを扱う考え方が重要になります。なお、こうした分析結果は、特定企業のサービス利用データに基づくものであり、すべての職場や国の実態をそのまま表すものではありません。

四件目。7月7日、Metaは、画像生成モデルMuse Imageと、動画生成モデルMuse Videoのプレビューを発表しました。Metaの発表では、画像生成の過程で検索やコード実行などのツールを使い、事実性や正確さを高める仕組みを説明しています。

ここで押さえてたいのは、Embeddingやベクトルは文章だけの技術ではないということです。画像、音声、動画も特徴をベクトルに変換できます。将来的には、「この商品の写真に似た画像を探す」「会議録音の中から特定の話題が出た箇所を探す」といった、複数の種類のデータを横断する検索がますます重要になります。

今日の要点を整理します。

一つ目。Embeddingとは、単語、文章、画像などの意味や特徴を、数字の並び、つまりベクトルに変換する技術です。

二つ目。意味が近いデータは、ベクトル空間の近い場所に配置されやすいため、言葉が完全に一致しなくても関連情報を探せます。

三つ目。ベクトル検索は意味検索が得意で、キーワード検索は型番や固有名詞などの正確な一致が得意です。実務では両方を組み合わせるハイブリッド検索が有効です。

四つ目。RAGでは、Embeddingを使って関連資料を探し、その資料をLLMのコンテキストに渡して回答させます。Embeddingは、LLMのために資料を探す司書のような役割です。

五つ目。実務で重要なのは、モデルの性能だけではありません。適切なチャンク分割、メタデータ、アクセス権、文書更新、評価、人による確認が必要です。

最後に、1分理解チェックです。答えは少し考えてから聞いてください。

第一問。Embeddingとは、何を何に変換する技術でしょうか。

答え。文章や単語、画像などの意味や特徴を、数字の並びであるベクトルに変換する技術です。

第二問。ベクトル検索が得意なのは、完全一致検索と、意味の近い情報を探す検索のどちらでしょうか。

答え。意味の近い情報を探す検索です。

第三問。RAGにおけるEmbeddingの主な役割は何でしょうか。

答え。質問に関連する文書や情報の断片を検索し、LLMへ渡す候補を選ぶことです。

第四問。長い文書を検索しやすいまとまりに分けることを何と呼ぶでしょうか。

答え。チャンク分割、またはチャンク化です。

第五問。社内検索で、Embeddingの精度だけでは不十分な理由を一つ挙げてください。

答え。たとえば、閲覧権限の管理、文書の更新日、公開範囲、正しい出典の表示、人による確認などが必要だからです。

今日は、AIが言葉を数字の地図に変え、その近さを利用して情報を探す仕組みを学びました。

次回、Day 4では、「Transformerの基本」に進みます。

LLMが、文章の中のどの言葉に注目し、どうやって長い文脈の関係を扱うのか。今日学んだトークン、コンテキスト、Embeddingが、Transformerの中でどのようにつながるのかを見ていきます。

それでは、AI Daily Academy、Day 3を終わります。

参考情報・一次情報

タイトル

How Claude 's text watermarking works

発行元

Anthropic

発表日

2026年8月14日

URL

<https://www.anthropic.com/news/claude-text-watermark>

タイトル

From assistance to execution: How enterprises put AI to work

発行元

OpenAI

発表日

2026年8月12日

URL

<https://openai.com/index/how-enterprises-put-ai-to-work/>

タイトル

Understanding the AI economy

発行元

Google

発表日

2026年7月23日

URL

<https://blog.google/innovation-and-ai/technology/research/understanding-the-ai-economy/>

タイトル

Introducing Muse Image and Muse Video

発行元

Meta AI

発表日

2026年7月7日

URL

<https://ai.meta.com/blog/introducing-muse-image-muse-video-msl/>